Food Store - La base de la tablet

TP Unidad 8 - Base de Datos II | Franco Rios y Santino Barone | Noviembre 2026

> **Regla de oro:** el móvil propone, el servidor dispone. La tablet solo consulta una instantánea y arma borradores. Precio, stock y ventas los decide PostgreSQL. **SIMULADO_APROBADO no es una venta.**

1. La base de la tablet

1.a ¿Qué datos viajan y qué datos no?

Viajan a la tablet	No viajan
`id` del producto, `nombre`, `categoria`, `precio_centavos`, `stock`, `disponible`, `eliminado` y `fecha_copia`. Son los datos mínimos para armar un borrador y explicar el resultado del semáforo.	Datos de `cliente` (nombre, email, teléfono), cuentas y `password_hash` de `usuario`, pedidos históricos, totales, formas de pago, direcciones, logs y cualquier dato personal.

La tablet no necesita saber quién compró ni cuánto se vendió: solo necesita cotizar y proponer. El precio y el stock son del servidor; si viajaran los pedidos históricos, se duplicaría información sensible y se podría intentar "confirmar" una venta desde un dispositivo sin señal. Por eso se aplica minimización de datos: viaja el catálogo operativo, no la historia comercial.

1.b Las cuatro tablas de la tablet

`schema_sqlite.sql` crea `producto_cache`, `pedido_borrador`, `item_borrador` y `cola_sync` con `PRAGMA foreign_keys = ON` y `STRICT`. Se agregan `snapshot_servidor` (instantánea inmutable de la corrida) y `snapshot_raw` (staging TEXT para aceptar los booleanos `t/f` de PostgreSQL).

Tabla	Reglas principales
`producto_cache`	PK; precio y stock enteros no negativos; `disponible` y `eliminado` solo 0/1; `fecha_copia`.
`pedido_borrador`	PK UUID; `forma_pago` solo EFECTIVO, TARJETA o TRANSFERENCIA; fecha.
`item_borrador`	FK a borrador y a producto_cache; PK compuesta; `cantidad > 0`; precio visto entero.
`cola_sync`	PK UUID (idempotencia); FK al borrador; estados PENDIENTE, CONFLICTO, RECHAZADO, SIMULADO_APROBADO; motivo.

1.c Pruebas SQLite

- **C1:** `PRAGMA database_list` muestra solo `main`; la consulta del catálogo devuelve 13 productos visibles sin conectarse al servidor.
- **Borrador completo:** `BEGIN ... COMMIT` con cabecera, 2 items y 1 solicitud.
- **Borrador descartado:** `ROLLBACK` deja 0 filas en las tres tablas.
- **C7a:** cantidad 0 falla por `CHECK constraint failed: cantidad > 0`; 0 filas.
- **C7b:** producto 999999 falla por `FOREIGN KEY constraint failed`; 0 filas.

Evidencia: `docs/evidencias/04_pruebas_sqlite.txt`.

2.a La instantánea

La vista `v_snapshot_movil` filtra la parada 5 e incluye `fecha_copia` calculada con `to_char(now(), ...)`. Se exporta con `\copy` a `movil/servidor_snapshot.csv` (16 filas). En SQLite se importa a `snapshot_raw` y `importar_snapshot.sql` convierte `t/f` a 1/0, reemplaza `snapshot_servidor` y actualiza `producto_cache` con upsert. La fecha de la copia queda en cada fila y en la evidencia.

2.b El semáforo

`semaforo.sql` evalúa solo solicitudes `PENDIENTE` y escribe estado y motivo en `cola_sync`. Precedencia: **rojo** (producto inexistente, dado de baja, no disponible, stock 0 o cantidad mayor al stock) → `RECHAZADO`; **amarillo** (no hay rojo pero cambió un precio) → `CONFLICTO`; **verde** (todo coincide) → `SIMULADO_APROBADO`. Repetir la simulación no cambia filas ya evaluadas porque el `UPDATE` filtra por `estado = 'PENDIENTE'`. No se escribe nada en PostgreSQL.

2.c Los casos C2 a C6

Caso	Qué se preparó	Resultado	Explicación
C2	Borrador con leche a 95000; el servidor la sube a 105000.	`CONFLICTO`; luego de aceptar, nueva solicitud `SIMULADO_APROBADO`.	El precio visto no coincide. No se cambia nada en PostgreSQL. El cliente acepta: se actualiza el precio del borrador y se crea una solicitud nueva con otro UUID.
C3	Borrador con 5 manzanas; hay 4.	`RECHAZADO`	La cantidad pedida supera el stock de la instantánea: `id 12: pedidas 5, stock 4`.
C4	Yerba mate vigente; después el servidor la marca `eliminado = 1`.	`RECHAZADO`; el item local queda.	El catálogo deja de mostrar el producto, pero el item se conserva para explicar el rechazo. La instantánea incluye las bajas lógicas.
C5	Borrador sin cambios.	`SIMULADO_APROBADO`	Precio, stock y disponibilidad coinciden. No se crea ningún pedido.
C6	Reenviar la misma solicitud (mismo UUID).	Duplicado rechazado; una sola fila.	La PK de `cola_sync` protege la idempotencia en la tablet; el semáforo no re-evalúa porque ya no está PENDIENTE.

> **Verificación final en PostgreSQL:** `pedidos_base=3, pedidos=3, detalles_base=4, detalles=4, stock_base=295, stock=295` → **VERDE: pedidos y stock sin cambios.** La simulación no creó pedidos, no descontó stock y no modificó precios de pedidos ya confirmados.

¿Dónde iría la protección real del reenvío? (parada 8)

En el servidor: una clave única de idempotencia (`pedido.id_local UUID UNIQUE`) o una tabla `solicitud_sync` con UUID único, y la confirmación de la venta dentro de una transacción que descuente stock con `UPDATE ... WHERE stock >= cantidad`. La tablet puede tener su PK, pero la protección definitiva no puede depender del dispositivo: si se pierde o se reinstala, el servidor debe reconocer el UUID repetido. La tablet nunca confirma ventas.

3. Integrar y revisar

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda sobre cada dato
PostgreSQL servidor	Producto, precio, stock, disponibilidad, bajas lógicas, pedidos confirmados y detalle con precio copiado. También las cuentas y los roles de la Unidad 5.	Es la **fuente de verdad**. Decide precio, stock y venta. La tablet solo lee su instantánea. Ningún otro motor descuenta stock.
MongoDB Unidad 7	Documentos `pedidos` con `lineas_pedido` embebidas: una foto denormalizada del pedido para consultas y lectura rápida.	No decide stock ni precio. Es un **modelo de lectura/histórico**; el precio copiado conserva el valor de la venta aunque el catálogo cambie. PostgreSQL sigue siendo el dueño de la transacción.
SQLite tablet	`producto_cache`, `snapshot_servidor`, borradores, items y `cola_sync`.	Es una **copia de trabajo offline**. No modifica precio ni stock. El servidor dispone; la tablet solo propone y muestra el semáforo.

Flujo: PostgreSQL → `v_snapshot_movil` → CSV → SQLite (`snapshot_raw` → `snapshot_servidor` → `producto_cache`) → borrador → `cola_sync`. La vuelta (confirmar venta) queda fuera del TP y, en un sistema real, pasaría por una API transaccional del servidor.

3.b Plan de mantenimiento (agregado al de la Unidad 6)

Respaldo y prueba de restauración

Objeto	Frecuencia	Cómo se prueba
PostgreSQL (servidor)	Diario 02:00, `pg_dump -Fc`; retención 7 diarios + 4 semanales + 3 mensuales.	Restauración semanal en base aislada; conteos, integridad referencial, totales y **hash de contenido** (md5 de filas ordenadas), no solo conteos.
Tablet SQLite	Al cierre de cada turno o antes de actualizar la app; copia con `VACUUM INTO` o `.backup`.	Abrir la copia en DB Browser, correr `PRAGMA integrity_check` y comparar cantidad de borradores/pendientes.
Snapshot	Cada vez que hay señal; se conserva el CSV de la última corrida.	Comparar `fecha_copia` y cantidad de filas; si no coincide, reexportar.

En la prueba de la Unidad 6 se restauró la base canónica: 20.000 clientes, 50.000 productos, 200.000 pedidos y 399.997 detalles, 8/8 controles en 0, RTO 6,25 s y RPO 0 h. Además se calculó el md5 de `producto(id, precio, stock)` en origen y en la restaurada: ambos dieron `5a50feb330f8485a1e809d1178a3d80a`. Ese hash debería incorporarse al script para detectar corrupción que no cambia conteos.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

- La tablet sigue operando offline con su instantánea; no se bloquea la feria.
- Al reconectar, compara `fecha_copia`. Si es anterior al respaldo restaurado, **no sube** borradores automáticamente: primero descarga una instantánea nueva y vuelve a correr el semáforo.
- Los borradores locales se conservan; los que estaban en conflicto se revisan. Nunca se aplica stock local sobre el servidor restaurado.
- El servidor mantiene la clave de idempotencia para no duplicar ventas que sí alcanzaron a confirmarse antes de la restauración.
- Se registra el incidente y se revisa la diferencia entre pedidos del servidor y solicitudes locales.

¿Cada cuánto y quién revisa CONFLICTO o RECHAZADO?

Quién	Cuándo	Qué hace
Supervisor de turno	Al cierre de cada turno (diario)	Revisa la cola de la tablet: precios cambiados, bajas y stock insuficiente. Decide si el cliente acepta el precio nuevo o se anula el borrador. Deja registro en `cola_sync.motivo`.
Administrador de datos (`admin_datos`)	Semanal	Audita patrones repetidos (mismo producto, mismo precio), valida que las bajas lógicas estén bien y ajusta el catálogo. No ve datos personales innecesarios.
Auditor de seguridad	Mensual	Revisa accesos y cambios de precio; controla que ningún rol de aplicación pueda modificar la tabla de usuarios.

SLA propuesto: todo `CONFLICTO` o `RECHAZADO` se revisa dentro de las 24 h. Un rechazo por stock no se "fuerza": se corrige el stock en el servidor o se reduce la cantidad. Un conflicto de precio no se resuelve desde la tablet: se acepta el precio del servidor o se anula el borrador.

3.c Code review con IA

Se revisaron los scripts de TP1 a TP8 con OpenCode en modo Plan. La tabla registra cada hallazgo, la decisión y la comprobación. Hay cinco unidades distintas representadas y un hallazgo descartado con fundamento.

Unidad / archivo	Hallazgo	Decisión	Motivo	Cómo se comprobó
U3 TP5 schema.sql	`producto.precio` permite 0 (`>= 0`), pero `detalle_pedido.precio_unitario` exige `> 0`.	Aceptado	Un producto a precio 0 puede existir en el catálogo y nunca se puede vender. Conviene alinear ambos checks a `> 0`.	En `copia_trabajo`: inserté producto con precio 0 (OK) y detalle con precio 0 → `detalle_pedido_precio_unitario_check`; rollback. `SELECT count(*) FROM producto WHERE precio=0` = 0.
U3 TP5 schema.sql	`CHECK (fecha <= now())` usa una función no inmutable y bloquea pedidos futuros.	Aceptado	Es una regla de negocio válida para ventas presenciales, pero complica una restauración con reloj anterior y una programación futura. Debe quedar documentado.	Inserté un pedido con `now() + interval '1 day'` → `pedido_fecha_check`; rollback. La restauración de la U6 no falló porque las fechas eran históricas.
U5 roles.sql	Los roles de login se crean con la contraseña literal `'CAMBIO_OBLIGATORIO'`.	Aceptado	Aunque está documentado como placeholder, una contraseña literal en el repositorio es un riesgo si se despliega sin rotarla.	Revisión del script y de `pg_authid.rolpassword` (hash presente). En producción: crear el rol sin password y usar `\password` o un secreto externo.
U6 restore_test.ps1	Valida conteos y 8 invariantes, pero no compara un hash de contenido.	Aceptado	Un dato corrupto con el mismo conteo podría pasar. Hay que agregar un checksum por tabla.	Restauración real: 8/8 OK. Calculé md5 de `producto(id,precio,stock)` en origen y restaurada: `5a50feb330f8485a1e809d1178a3d80a` en ambas.
U6 TP6 trigger resumen	`fn_resumen_aplicar` lanza excepción si falta una celda para un delta negativo y borra celdas en 0.	Descartado	La excepción es una **alarma deliberada** de inconsistencia. Silenciarla con un upsert ocultaría una desincronización que la auditoría debe detectar. El borrado en 0 mantiene el resumen minimal.	La auditoría de TP6 devolvió 0 filas y `suma_global_coincide = t`; la prueba de sincronización mantuvo el resumen. Cambiarlo no mejora la integridad.
U7 consultas_mongodb.js	La consulta por `lineas_pedido.producto_id` no tiene índice multikey.	Aceptado	Con muchos pedidos, el filtro recorre documentos. Conviene `createIndex({ "lineas_pedido.producto_id": 1 })`.	Revisión estática: no hay `mongosh` en el equipo. Queda como recomendación para cuando se levante MongoDB; se verificaría con `explain("executionStats")`.
U8 semaforo.sql	La primera versión duplicaba el motivo de stock 0: "sin stock" y "stock insuficiente".	Corregido	El mensaje al vendedor debe ser único y claro; se agregó `stock_servidor > 0` a la condición de insuficiente.	Producto 13 (stock 0) quedó `RECHAZADO` con motivo `sin stock: id 13 sin stock`; producto 11 (10 vs 8) quedó con `stock insuficiente`.

> **Hallazgo descartado con fundamento (U6):** la IA propuso reemplazar el `RAISE EXCEPTION` del trigger de resumen por un upsert silencioso. Se descartó porque el objetivo del trigger es justamente detectar que la fuente de verdad y el resumen se separaron; la auditoría de TP6 exige 0 filas y suma global coincidente. Un upsert ocultaría el problema en lugar de resolverlo.

4. Kiro, bitácora y defensa

4.a Spec en Kiro

`docs/kiro/requirements.md` contiene cuatro requisitos EARS: REQ-01 precio distinto (C2), REQ-02 stock insuficiente (C3), REQ-03 producto inexistente/no disponible/dado de baja (C4 y C7b) y REQ-04 idempotencia del reenvío (C6). `design.md` describe la arquitectura, el modelo, el flujo y las decisiones descartadas; `tasks.md` baja los requisitos a tareas verificables. La spec se escribió antes de cerrar el código y se ajustó después de probar los casos.

Requisito	Enunciado resumido	Caso	Evidencia
REQ-01	Si el precio visto difiere, marcar CONFLICTO.	C2	`casos_c2_c6.txt`
REQ-02	Si la cantidad supera el stock, RECHAZADO.	C3	`casos_c2_c6.txt`
REQ-03	Si no existe, no está disponible o está de baja, RECHAZADO; conservar item.	C4 / C7b	`casos_c2_c6.txt` y `04_pruebas_sqlite.txt`
REQ-04	Mismo UUID: rechazar duplicado y no re-evaluar.	C6	`casos_c2_c6.txt`

4.b Bitácora de IA y DUIA

`docs/bitacora_ia.md` tiene siete entradas con el prompt, lo generado, lo aceptado, lo descartado y la verificación. `docs/DUIA.md` declara herramientas, modelos, alcance, datos inventados y responsabilidad. Ejemplos de descartes: borrar items locales al dar de baja un producto, reutilizar el UUID de una solicitud y afirmar en el informe que la tablet "confirma ventas".

4.c README y ensayo de defensa (10 minutos)

`README.md` permite recrear todo desde cero: crear `foodstore_u8`, ejecutar esquema y datos, exportar el CSV, crear la base SQLite, importar, correr pruebas y semáforo, y repetir C2-C6.

Minuto	Qué se muestra
0-1	Regla de oro y arquitectura de tres motores. Aclarar que SIMULADO_APROBADO no es venta.
1-3	CSV y `producto_cache`: qué viaja y qué no. C1 sin servidor.
3-5	Abrir `foodstore_movil.db`, correr `semaforo.sql` y mostrar verde/amarillo/rojo.
5-7	C2 en vivo: cambiar precio en PostgreSQL, exportar, importar, mostrar CONFLICTO y luego la aceptación.
7-8	C3, C4 y C6 en la evidencia: stock, baja lógica y UUID repetido.
8-9	`05_verificacion.sql`: 3 pedidos, 4 detalles, stock 295, VERDE.
9-10	Un hallazgo del code review (el trigger de TP6 descartado) y cierre: la tablet propone, el servidor dispone.

> **Checklist de entrega:** `food-store/` con README, `movil/` (schema, pruebas, semáforo, CSV, .db), `docs/` (informe PDF, kiro, bitácora, DUIA) y evidencia. Subir con etiqueta `tp8` antes del cierre de la semana 13.